Section 1.1: Linear Regression

AI Class — Lecture Notes [Student Copy]

---

1 Problem Setup

Suppose we observe n data points (x_i, y_i) and believe there is an approximately linear relationship between x and y, corrupted by noise. For example:

x	1	2	3	4
y	1	3	3	5

We want to find a line

$$\hat{y} = wx + b$$

that best fits this data, where w is the **slope (weight)** and b is the **intercept (bias)**.

Because of noise, no single line passes through all four points exactly. We therefore need a way to quantify *how wrong* our predictions are, so that we can find the *least wrong* line.

2 Cost Function: Mean Squared Error

For a given choice of w and b, the **residual** at point i is

$$e_i = y_i - \hat{y}_i = y_i - wx_i - b.$$

We measure the overall fit using the **Mean Squared Error (MSE)**:

$$J(w, b) = \_\_\_\_\_ \tag{1}$$

Several properties make MSE a natural choice:

- Squaring makes all errors positive, so positive and negative residuals do not cancel.
- Larger errors are penalised more than smaller ones (quadratic penalty).
- $J = 0$ is a perfect fit; otherwise the larger J, the worse the fit.
- $J(w, b)$ is a smooth, bowl-shaped (convex) surface in the (w, b) plane, guaranteeing a unique global minimum.

Goal: find the values of w and b that minimise $J(w, b)$.

3 Minimising MSE: Scalar Calculus Derivation

Because J is smooth and convex, the minimum occurs where both partial derivatives vanish simultaneously:

$$\frac{\partial J}{\partial b} = 0 \qquad \text{and} \qquad \frac{\partial J}{\partial w} = 0.$$

This gives two equations in two unknowns, which we solve sequentially.

3.1 Step 1: Solve for b

Differentiating (1) with respect to b:

$$\frac{\partial J}{\partial b} = \frac{1}{n}\sum_{i=1}^{n} 2(y_i - wx_i - b)(-1) = \frac{-2}{n}\sum_{i=1}^{n}(y_i - wx_i - b) = 0.$$

Expanding and dividing by n:

$$\frac{1}{n}\sum_{i=1}^{n} y_i = w \cdot \frac{1}{n}\sum_{i=1}^{n} x_i + b \quad \Longrightarrow \quad \bar{y} = w\bar{x} + b.$$

$$b = \_\_\_\_\_$$

Interpretation. For any slope w, the optimal line always passes through the *centroid* $(\bar{x}, \bar{y})$ of the dataset.

3.2 Step 2: Solve for w

Differentiating (1) with respect to w:

$$\frac{\partial J}{\partial w} = \frac{-2}{n}\sum_{i=1}^{n}(y_i - wx_i - b)\, x_i = 0.$$

Substituting $b = \bar{y} - w\bar{x}$ and letting $\tilde{x}_i = x_i - \bar{x}$, $\tilde{y}_i = y_i - \bar{y}$:

$$\sum_{i=1}^{n}(y_i - wx_i - \bar{y} + w\bar{x})\, x_i = 0 \quad \Longrightarrow \quad \sum_{i=1}^{n}\left(\tilde{y}_i - w\tilde{x}_i\right) x_i = 0.$$

Expanding and noting that $\sum_i \tilde{x}_i = 0$:

$$\sum_{i=1}^{n} \tilde{y}_i\, x_i = w\sum_{i=1}^{n} \tilde{x}_i\, x_i \quad \Longrightarrow \quad \sum_{i=1}^{n} \tilde{x}_i\tilde{y}_i = w\sum_{i=1}^{n} \tilde{x}_i^2.$$

$$w = \_\_\_\_\_$$

Interpretation. The optimal slope equals the *sample covariance* of x and y divided by the *sample variance* of x. A positive covariance (both tend to increase together) yields a positive slope, and vice versa.

3.3 Numerical Result

For the dataset above, $\bar{x} = 2.5$ and $\bar{y} = 3$:

i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$
1	-1.5	-2	3.0	2.25
2	-0.5	0	0.0	0.25
3	0.5	0	0.0	0.25
4	1.5	2	3.0	2.25
		Sum	6	5

$w =$ _____, $b =$ _____.

The fitted line is $\hat{y} =$ _____, with:

$$\text{MSE} = \frac{(-0.2)^2 + (0.6)^2 + (-0.6)^2 + (0.2)^2}{4} = \text{_____} \neq 0.$$

The non-zero MSE reflects irreducible noise in the data — not a failure of the method.

4 Generalisation: Multiple Features

With a single feature we derived two scalar equations. In practice, a data point often has d features $x_1, x_2, \ldots, x_d$, and the model becomes:

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b.$$

Taking a partial derivative for each of the $d+1$ parameters and setting them to zero would produce $d+1$ scalar equations with the same structure. Rather than repeating this process, we rewrite everything in **matrix form**, which handles all parameters simultaneously.

4.1 Matrix Notation

Collect all parameters into a single vector:

$$\boldsymbol{\theta} = \begin{bmatrix} b \\ w_1 \\ \vdots \\ w_d \end{bmatrix} \in \mathbb{R}^{d+1}.$$

Stack all n data points row-by-row into the **design matrix** $\mathbf{X} \in \mathbb{R}^{n \times (d+1)}$, prepending a column of ones to account for the bias:

$$\mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \cdots & x_d^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \cdots & x_d^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & x_2^{(n)} & \cdots & x_d^{(n)} \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}.$$

With this notation, the vector of all predictions is simply $\mathbf{X}\boldsymbol{\theta}$, and the MSE cost becomes:

$$J(\boldsymbol{\theta}) = \_\_\_\_\_$$

4.2 The Normal Equations

Setting the gradient of J with respect to $\boldsymbol{\theta}$ to zero and expanding step by step:

$$\begin{aligned}
&\nabla_{\boldsymbol{\theta}} J = \mathbf{0} \\
\Rightarrow\ &\nabla_{\boldsymbol{\theta}} \frac{1}{n} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 = \mathbf{0} \\
\Rightarrow\ &\nabla_{\boldsymbol{\theta}} \left(\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\right)^\top \left(\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\right) = \mathbf{0} \\
\Rightarrow\ &\nabla_{\boldsymbol{\theta}} (_____) = \mathbf{0} \\
\Rightarrow\ &_____ = \mathbf{0} \\
\Rightarrow\ &\mathbf{X}^\top \mathbf{X}\, \boldsymbol{\theta} = \mathbf{X}^\top \mathbf{y}.
\end{aligned}$$

Provided $\mathbf{X}^\top\mathbf{X}$ is invertible, the unique solution is:

$$\boldsymbol{\theta} = \_\_\_\_\_$$

This is the **Normal Equation** — a single closed-form formula that solves linear regression for any number of features d and any number of data points n (provided $n > d + 1$ and the columns of $\mathbf{X}$ are linearly independent).

5 Example: Two Binary Features

Example 5.1. Consider a dataset with two binary inputs where the true relationship is $y = x_1 + x_2$:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	2

The design matrix and target vector are:

$$\mathbf{X} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 2 \end{bmatrix}.$$

Computing:

$$\mathbf{X}^\top\mathbf{X} = \begin{bmatrix} 4 & 2 & 2 \\ 2 & 2 & 1 \\ 2 & 1 & 2 \end{bmatrix}, \qquad \mathbf{X}^\top\mathbf{y} = \begin{bmatrix} 4 \\ 3 \\ 3 \end{bmatrix}.$$

Solving $(\mathbf{X}^\top\mathbf{X})\,\boldsymbol{\theta} = \mathbf{X}^\top\mathbf{y}$ gives:

---

so $\hat{y} =$ _____ with MSE = _____.

The normal equations recover the exact underlying relationship because the data is truly linear.

Looking ahead. In Section 1.3 we will use the same four input points $\{(0,0), (0,1), (1,0), (1,1)\}$ but change the output labels to $\{0, 1, 1, 0\}$ (the XOR function). Applying the normal equations to that dataset yields $w_1 = w_2 = 0$, $b = \frac{1}{2}$ — the model simply predicts the mean 0.5 everywhere, achieving MSE = 0.25. This is the best any linear model can do on XOR, which motivates the need for non-linearity.

6 Summary

Model	$\hat{y} = \boldsymbol{\theta}^\top\mathbf{x}$
Loss function	MSE: $J = \frac{1}{n}\|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2$
Closed-form solution	Normal equations: $\boldsymbol{\theta} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$
Works well when	The true relationship is (approximately) linear

Up next (Section 1.2): What if the output y is a *category* rather than a continuous number? Predicting a probability requires a different model and a different loss function — logistic regression with cross-entropy.